

RELATÓRIO — AULA 02

Introdução ao Cálculo Numérico e Erros Computacionais

Disciplina: Cálculo Numérico

Curso: Bacharelado em Ciência e Tecnologia — UNIFESP

Aluna: Victória Barbosa

Turma: NB

Aula: 02

Ambiente: Google Colab / Python

Introdução

Nesta aula, começamos a estudar de forma prática os principais conceitos de Cálculo Numérico, com foco no uso de aproximações e na análise dos erros associados aos resultados. Eu compreendi que nem todo problema matemático possui uma solução analítica simples ou viável de ser encontrada apenas por manipulações matemáticas. Nesses casos, os métodos numéricos permitem obter soluções aproximadas por meio de procedimentos computacionais, tornando possível trabalhar com problemas mais complexos de forma prática.

Um dos pontos que considerei mais importantes foi perceber que obter um resultado numérico não é suficiente: também é necessário avaliar sua precisão e entender o quanto ele pode estar distante de um valor de referência. Durante a aula, utilizei Python e Google Colab para aplicar esses conceitos em diferentes exemplos, observando diretamente os erros e suas consequências nos resultados. Dessa forma, consegui relacionar a parte teórica com a prática e compreender melhor como as aproximações, a precisão e as limitações da representação numérica podem influenciar os cálculos.

1. Preparação do ambiente

Para iniciar as atividades, preparei o ambiente no Google Colab utilizando as bibliotecas `math`, `numpy` e `matplotlib`. A biblioteca `math` foi utilizada para funções e constantes matemáticas, como π e $\sqrt{2}$; a `numpy` para operações numéricas e representação de números em ponto flutuante; e a `matplotlib` para a criação dos gráficos utilizados durante a aula.

```
[1]
✓ Os # Importa a biblioteca de funções matemáticas da linguagem Python.
import math

# Importa a biblioteca NumPy e cria o apelido convencional np.
import numpy as np

# Importa o módulo de gráficos pyplot e cria o apelido plt.
import matplotlib.pyplot as plt

# Define um estilo visual claro para todos os gráficos do notebook.
plt.style.use("seaborn-v0_8-whitegrid")

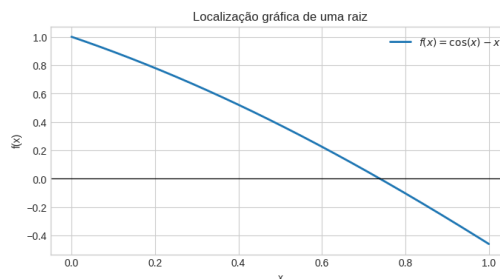
# Exibe uma mensagem para confirmar que a célula foi executada.
print("Ambiente preparado com sucesso!")

*** Ambiente preparado com sucesso!
```

Essa etapa foi simples, mas importante para que eu entendesse as principais ferramentas que seriam utilizadas ao longo da disciplina. Também pude perceber como o Python funciona como uma ferramenta de apoio ao Cálculo Numérico, permitindo realizar cálculos, analisar resultados e visualizar graficamente os problemas estudados.

2. Por que precisamos do Cálculo Numérico?

Estudei a necessidade dos métodos numéricos a partir da função $f(x) = \cos(x) - x$ – $x = 0$, cuja raiz não é simples de encontrar por manipulações algébricas. Por isso, métodos numéricos podem ser utilizados para obter uma aproximação da solução.



Ao analisar o gráfico, observei que a curva cruza o eixo horizontal entre 0 e 1, indicando que a raiz está aproximadamente nesse intervalo. Com isso, entendi que o gráfico ajuda a localizar a solução, mas não determina sozinho seu valor com precisão.

Discussão guiada

Analise o gráfico antes de responder:

1. Em qual intervalo a raiz parece estar?

Resposta: A raiz parece estar no intervalo $[0.5; 1.0]$, pois é nessa região que o gráfico da função $f(x) = \cos(x) - x$ cruza o eixo x, indicando que $f(x)$ se aproxima de zero.

3. Valor de referência e aproximação

Em seguida, estudamos a diferença entre um valor de referência e uma aproximação, utilizando o número π como exemplo. O valor de `math.pi` foi considerado como referência e 3,14 como aproximação.

```
[ ] # Calcula a distância entre o valor de referência e a aproximação.
    erro_absoluto = abs(valor_referencia - valor_aproximado)

    # Divide o erro absoluto pelo módulo do valor de referência.
    erro_relativo = erro_absoluto / abs(valor_referencia)

    # Converte o erro relativo em porcentagem.
    erro_percentual = 100.0 * erro_relativo

    # Exibe o erro absoluto com dez casas decimais.
    print(f"Erro absoluto: {erro_absoluto:.10f}")

    # Exibe o erro relativo em notação científica.
    print(f"Erro relativo: {erro_relativo:.6e}")

    # Exibe o erro percentual com seis casas decimais.
    print(f"Erro percentual: {erro_percentual:.6f}%")

▼ Erro absoluto: 0.0015926536
  Erro relativo: 5.069574e-04
  Erro percentual: 0.050696%
```

Entendi que, na prática, nem sempre trabalhamos com um valor exato, mas com uma referência considerada suficientemente precisa. Como 3,14 possui menos casas decimais que π , existe uma diferença entre os valores. Essa diferença pode ser medida por meio dos erros absoluto, relativo e percentual, que foram os próximos conceitos estudados.

4. Erro absoluto, erro relativo e erro percentual

Nesta parte, aprendemos três formas de avaliar a diferença entre um valor de referência e uma aproximação. O erro absoluto mostra quanto os valores diferem, o erro relativo relaciona essa diferença ao valor de referência e o erro percentual apresenta essa relação em porcentagem. Entendi que o erro relativo é especialmente útil para avaliar a importância da diferença em relação ao tamanho do valor analisado.

Para a aproximação 3,14 de π , obtive erro absoluto de 0,0015926536, erro relativo de 0,0005069574 e erro percentual de 0,050696%. Com isso, percebi que, apesar de existir uma diferença entre os valores, ela é pequena em relação ao valor de π .

✈ **Observação:** O código e os resultados completos estão disponíveis no [Google Colab - Seção 4: Erro absoluto, erro relativo e erro percentual](#).

Atividade de interpretação

Use os resultados apresentados pela célula anterior para responder:

1. Qual é o erro absoluto da aproximação 3,14 para π ?

Resposta: O erro absoluto é aproximadamente 0,0015926536.

2. Qual é o erro relativo?

Resposta: O erro relativo é aproximadamente 0,0005069574.

3. Qual é o erro percentual?

Resposta: O erro percentual é aproximadamente 0,050696%.

4. O erro absoluto e o erro relativo possuem a mesma unidade?

Resposta: Não. O erro absoluto possui a mesma unidade da grandeza analisada, enquanto o erro relativo não possui unidade, pois é obtido pela divisão do erro absoluto pelo valor de referência e as unidades se cancelam.

Exemplo 1: A importância da escala do problema

Depois de estudar os tipos de erro, analisamos dois casos que mostraram por que o erro relativo é importante. Nos dois exemplos, o erro absoluto foi 1, mas os valores de referência eram diferentes: 1000 e 2.

```
[1] # Cria uma lista com pares formados por referência e aproximação.
    casos = [(1000.0, 999.0), (2.0, 1.0)]

    # Percorre cada par armazenado na lista.
    for referencia, aproximacao in casos:
        # Calcula o erro absoluto do caso atual.
        erro_abs = abs(referencia - aproximacao)

        # Calcula o erro relativo do caso atual.
        erro_rel = erro_abs / abs(referencia)

        # Exibe os valores e o erro relativo em porcentagem.
        print(f"referencia:g → {aproximacao:g}: Ea={erro_abs:g}; E%={100*erro_rel:.1f}%")

    ... 1000 → 999: Ea=1; E%=0.1%
        2 → 1: Ea=1; E%=50.0%
```

No primeiro caso, o erro representa apenas 0,1% do valor de referência, enquanto no segundo representa 50%. Com isso, entendi que o mesmo erro absoluto pode ter importâncias muito diferentes dependendo da escala do problema. Por isso, o erro relativo permite uma comparação mais adequada da precisão.

5. Algarismos significativos

Também estudamos o conceito de algarismos significativos, que está relacionado à precisão de uma medida ou aproximação. Um exemplo apresentado foi o número 0,00450, que possui três algarismos significativos, pois os zeros antes do primeiro algarismo diferente de zero não são considerados.

Também comparei aproximações sucessivas de $\sqrt{2}$, utilizando diferentes quantidades de casas decimais.

```
[1] # Cria diferentes aproximações para a raiz quadrada de 2.
    aproximacoes = [1.4, 1.41, 1.414, 1.4142]

    # Calcula um valor de referência para a raiz quadrada de 2.
    referencia_raiz = math.sqrt(2.0)

    # Percorre todas as aproximações da lista.
    for aproximacao in aproximacoes:
        # Calcula o erro relativo de cada aproximação.
        erro_rel = abs(referencia_raiz - aproximacao) / abs(referencia_raiz)

        # Exibe a aproximação e seu erro relativo.
        print(f"Aproximação={aproximacao}<7 | erro relativo={erro_rel:.3e}")

    Aproximação=1.4      | erro relativo=1.005e-02
    Aproximação=1.41    | erro relativo=2.979e-03
    Aproximação=1.414   | erro relativo=1.510e-04
    Aproximação=1.4142  | erro relativo=9.590e-06
```

Ao comparar os resultados, percebi que aumentar as casas decimais geralmente aproxima o valor calculado do valor de referência e reduz o erro relativo. Assim, compreendi melhor a relação entre precisão, algarismos significativos e erro numérico.

6. Representação de números no computador

Um conceito importante que estudamos foi a forma como o computador representa números reais. Como ele possui uma quantidade finita de bits, alguns números decimais não podem ser representados exatamente em binário. Isso pode gerar pequenas diferenças nos resultados, como observei ao comparar $0.1 + 0.2$ com 0.3 .

```
# Armazena a soma que desejamos testar.
a = 0.1 + 0.2

# Armazena o valor esperado.
b = 0.3

# Define uma tolerância absoluta pequena.
tolerancia = 1e-12

# Realiza uma comparação manual baseada na diferença absoluta.
comparacao_manual = abs(a - b) < tolerancia

# Realiza uma comparação robusta com tolerâncias relativa e absoluta.
comparacao_isclose = math.isclose(a, b, rel_tol=1e-12, abs_tol=1e-12)

# Exibe o resultado da comparação manual.
print("Comparação manual:", comparacao_manual)

# Exibe o resultado obtido com math.isclose.
print("Com math.isclose:", comparacao_isclose)

Comparação manual: True
Com math.isclose: True
```

Com esse exemplo, compreendi que uma pequena diferença não significa necessariamente um erro no cálculo, mas pode ser consequência da **precisão finita** utilizada pelo computador.

Por esse motivo, também aprendi que não é adequado comparar números reais utilizando apenas `==`. Em cálculos numéricos, podemos utilizar uma **tolerância** para verificar se dois valores são suficientemente próximos. A função `math.isclose()` permite realizar essa comparação considerando tolerâncias absoluta e relativa.

Para conhecer melhor os limites do tipo `float64`, utilizei `np.finfo(np.float64)`, que apresenta características como o maior e o menor número representável e o epsilon de máquina.

```
# Obtém as propriedades numéricas do tipo float64.
info_float = np.finfo(np.float64)

# Exibe a precisão de máquina.
print("Epsilon de máquina:", info_float.eps)

# Exibe o maior número finito representável.
print("Maior float64:", info_float.max)

# Exibe o menor número normal positivo representável.
print("Menor float64 normal positivo:", info_float.tiny)

Epsilon de máquina: 2.220446049250313e-16
Maior float64: 1.7976931348623157e+308
Menor float64 normal positivo: 2.2250738585072014e-308
```

Essa parte me ajudou a entender que existe uma diferença entre a representação matemática dos números e a forma como eles são armazenados pelo computador, sendo importante considerar essas limitações ao realizar cálculos numéricos.

7. Arredondamento e truncamento

Nesta etapa, aprendemos a diferença entre arredondamento e truncamento. No arredondamento, procuro obter o valor mais próximo dentro da quantidade de casas desejada, observando o próximo algarismo para decidir se o último será alterado. Já no truncamento, os algarismos posteriores são simplesmente descartados.

Por exemplo, ao trabalhar com 3,14159 e duas casas decimais, ambos podem resultar em 3,14, mas por processos diferentes. Também compreendi que erro de truncamento pode ter outro significado no Cálculo Numérico, relacionado à substituição de um processo infinito por uma quantidade finita de etapas ou termos.

Assim, percebi que arredondamento e truncamento são procedimentos diferentes, mesmo que possam produzir resultados iguais em alguns casos.

★ **Observação:** Os exemplos e códigos utilizados para demonstrar esses procedimentos permanecem disponíveis no [Google Colab - Seção 7: Arredondamento e truncamento](#).

8. Propagação de erros

Estudamos também a propagação de erros, compreendendo que um pequeno erro em uma medida pode produzir um erro maior no resultado final. Para observar isso, utilizei o volume de uma esfera, dado por $V = (4/3)\pi r^3$, considerando um raio de referência de 1,000 cm e um raio medido de 1,010 cm.

```
[1] # Define o raio de referência em centímetros.
raio_referencia = 1.000

# Define um raio medido com erro de 1%.
raio_medido = 1.010

# Calcula o volume usando o raio de referência.
volume_referencia = (4.0 / 3.0) * math.pi * raio_referencia ** 3

# Calcula o volume usando o raio medido.
volume_medido = (4.0 / 3.0) * math.pi * raio_medido ** 3

# Calcula o erro relativo do raio.
erro_rel_raio = abs(raio_medido - raio_referencia) / raio_referencia

# Calcula o erro relativo do volume.
erro_rel_volume = abs(volume_medido - volume_referencia) / volume_referencia

# Exibe o erro percentual do raio.
print(f'Erro no raio: {100*erro_rel_raio:.3f}%')

# Exibe o erro percentual propagado para o volume.
print(f'Erro no volume: {100*erro_rel_volume:.3f}%')
```

*** Erro no raio: 1.000%
Erro no volume: 3.030%

O erro no raio foi de 1%, enquanto o erro no volume foi de aproximadamente 3,030%. Entendi que isso ocorre porque o raio está elevado ao cubo, fazendo com que o erro seja aproximadamente triplicado.

Questões para discussão

1. Qual foi o erro percentual introduzido na medida do raio?

Resposta: O erro percentual introduzido na medida do raio foi de 1%, pois o raio de referência é 1,000 cm e o raio medido é 1,010 cm.

2. Qual foi o erro percentual observado no volume?

Resposta: O erro percentual observado no volume foi de aproximadamente 3,030%.

3. Por que o erro no volume é aproximadamente três vezes maior?

Resposta: O erro no volume é aproximadamente três vezes maior porque o raio está elevado ao cubo na fórmula do volume da esfera: $V = (4/3)\pi r^3$. Pela aproximação linear, quando uma grandeza está elevada a uma potência nn , seu erro relativo é aproximadamente multiplicado por nn . Como o raio está elevado à terceira potência, um erro de aproximadamente 1% no raio produz aproximadamente 3% de erro no volume.

4. A aproximação linear fornece exatamente o mesmo resultado do cálculo completo?

Resposta: Não. A aproximação linear fornece uma estimativa de aproximadamente 3%, enquanto o cálculo completo resulta em aproximadamente 3,030%. Portanto, os resultados são próximos, mas não exatamente iguais.

9. Perda de significância por cancelamento numérico

Nesta etapa, estudei a perda de significância por cancelamento numérico, que ocorre quando números muito próximos são subtraídos e alguns algarismos significativos podem ser perdidos.

O exemplo utilizado foi uma expressão envolvendo raízes. Para valores grandes de x , os termos ficam muito próximos. Ao racionalizar a expressão, a subtração é eliminada, obtendo uma forma numericamente mais estável.

```
[1] # Define um valor grande de x para provocar cancelamento.
x_grande = 1e16

# Calcula a expressão diretamente por subtração.
forma_direta = math.sqrt(x_grande + 1.0) - math.sqrt(x_grande)

# Calcula a expressão equivalente na forma racionalizada.
forma_estavel = 1.0 / (math.sqrt(x_grande + 1.0) + math.sqrt(x_grande))

# Exibe o resultado da forma suscetível a cancelamento.
print("Forma direta:", forma_direta)

# Exibe o resultado da forma numericamente estável.
print("Forma estável:", forma_estavel)
```

Forma direta: 0.0
Forma estável: 5e-09

Com esse exemplo, compreendi que duas expressões podem ser matematicamente equivalentes, mas apresentar resultados diferentes quando calculadas pelo computador. Por isso, em Cálculo Numérico, também é importante considerar a estabilidade numérica da forma utilizada.

10. Atividade prática — Precisão das aproximações

Nesta atividade, comparei aproximações de $\sqrt{2}$, π e e , utilizando 1, 2, 4 e 6 casas decimais. Para cada aproximação, calculei o erro absoluto e o erro relativo, utilizando `round()` e as fórmulas estudadas anteriormente.

Ao analisar os resultados, percebi que, neste experimento, aumentar a quantidade de casas decimais tornou as aproximações mais próximas dos valores de referência e, conseqüentemente, reduziu os erros. Também observei que essa redução não ocorre exatamente da mesma forma para todas as constantes.

★ **Observação:** O código completo, incluindo o preenchimento dos `TODO` e os resultados obtidos, está disponível [no Google Colab - Seção 10: Atividade prática - precisão das aproximações](#).

Análises dos Resultados

1. *O erro sempre diminui quando mais casas decimais são mantidas?*

Resposta: Em geral, sim, neste experimento. Ao aumentar o número de casas decimais, as aproximações ficam mais próximas dos valores de referência e, conseqüentemente, os erros absoluto e relativo tendem a diminuir.

2. *A redução ocorre na mesma proporção em todos os casos?*

Resposta: Não. A redução do erro não ocorre necessariamente na mesma proporção para $\sqrt{2}$, π e e . Isso depende dos valores das constantes e de como ocorre o arredondamento em cada quantidade de casas decimais.

3. *Qual medida de erro é mais adequada para comparar números de magnitudes diferentes?*

Resposta: O erro relativo, porque ele considera o tamanho do valor de referência. Diferentemente do erro absoluto, permite comparar a importância do erro mesmo quando os valores possuem magnitudes diferentes.

11. Desafio - construção de uma função reutilizável

Nesta atividade, construí a função `calcular_erros()`, reunindo em um único código os cálculos de erro absoluto, relativo e percentual que estudei anteriormente.

A função também trata o caso em que o valor de referência é zero. Como o erro relativo envolveria uma divisão por zero, optei por retornar None, indicando que esse cálculo não está definido nesse caso.

```
[3]
✓ Os # Define uma função que receberá um valor de referência e uma aproximação.
def calcular_erros(referencia, aproximacao):
    # Calcula o erro absoluto.
    erro_abs = abs(referencia - aproximacao)

    # Verifica se o valor de referência é igual a zero.
    if referencia == 0.0:
        # O erro relativo não é definido quando a referência é zero.
        erro_rel = None
    else:
        # Calcula o erro relativo para uma referência diferente de zero.
        erro_rel = erro_abs / abs(referencia)

    # Converte o erro relativo em erro percentual.
    erro_percentual = 100.0 * erro_rel if erro_rel is not None else None

    # Devolve os três resultados calculados.
    return erro_abs, erro_rel, erro_percentual

# Teste 1: aproximação de pi.
print(calcular_erros(math.pi, 3.14))

# Teste 2: referência igual a zero.
print(calcular_erros(0.0, 0.01))

(0.0015926535897929917, 0.0005069573828972128, 0.05069573828972128)
(0.01, None, None)
```

Essa atividade me ajudou a entender a utilidade de criar funções reutilizáveis, evitando repetir os mesmos cálculos e tornando o código mais organizado.

Questão adicional - Referência igual a zero

Quando a referência é zero, a fórmula usual do erro relativo envolve uma divisão por zero. Qual seria uma decisão computacional adequada: retornar infinito, None, np.nan ou gerar uma exceção?

Resposta: Eu considero None uma decisão computacional adequada quando a referência é zero, pois o erro relativo não pode ser calculado pela fórmula usual devido à divisão por zero. O uso de None indica que não existe um valor válido para o erro relativo naquele caso, sem atribuir um resultado numérico que poderia ser interpretado incorretamente. Essa foi também a abordagem utilizada na função calcular_erros.

12. Fechamento e avaliação rápida

1. *Por que um resultado numérico deve vir acompanhado de uma estimativa de erro?*

Resposta: Um resultado numérico deve ser acompanhado de uma estimativa de erro porque uma aproximação pode não ser exatamente igual ao valor de referência. A estimativa de erro permite avaliar a precisão e a confiabilidade do resultado obtido.

2. *Quando o erro relativo é mais informativo do que o erro absoluto?*

Resposta: O erro relativo é mais informativo quando queremos comparar erros associados a valores de magnitudes diferentes, pois ele considera o tamanho do valor de referência. Dessa forma, permite avaliar a importância do erro proporcionalmente ao valor analisado.

3. *Por que números do tipo float não devem ser comparados apenas com ==?*

Resposta: Números do tipo float não devem ser comparados apenas com == porque muitos números decimais não possuem representação binária exata no computador. Pequenas diferenças podem surgir devido à precisão finita da representação. Por isso, é mais adequado utilizar uma tolerância, como na função `*math.isclose()`.

4. *Qual é a diferença entre erro de arredondamento e erro de truncamento?*

Resposta: O erro de arredondamento ocorre quando um número é substituído por outro valor mais próximo dentro da precisão desejada. Já o truncamento consiste em descartar os algarismos após determinada posição, sem necessariamente escolher o valor mais próximo. Além disso, em Cálculo Numérico, o termo erro de truncamento também pode representar a substituição de um processo infinito por uma quantidade finita de termos ou etapas.

5. *O que caracteriza a perda de significância por cancelamento numérico?*

Resposta: A perda de significância por cancelamento numérico ocorre quando dois números muito próximos são subtraídos, fazendo com que algarismos significativos sejam perdidos e podendo reduzir a precisão do resultado. Uma forma matematicamente equivalente da expressão pode ser mais estável numericamente e evitar esse problema.

Tarefa - Aplicação dos conceitos de erro

Para finalizar a aula, apliquei os conceitos estudados em uma situação de engenharia, escolhendo a temperatura de um processo industrial. Considerei 100 °C como valor de referência e utilizei duas aproximações: 99 °C e 99,8 °C. Para cada uma, calculei o erro absoluto, relativo e percentual utilizando a função `calcular_erros()` que desenvolvi anteriormente.

Os resultados mostraram que a aproximação de 99 °C apresentou erro absoluto de 1 °C e erro percentual de 1%, enquanto a aproximação de 99,8 °C apresentou erro absoluto de 0,2 °C e erro percentual de 0,2%. Portanto, concluí que 99,8 °C é a aproximação mais adequada, pois apresenta menor erro e maior precisão em relação ao valor de referência.

✦ **Observação:** O código completo da aplicação, os cálculos, a tabela de resultados e a saída do programa podem ser consultados no **Google Colab - Seção Tarefa**, onde a atividade foi desenvolvida e executada.

13. Considerações finais

A Aula 02 foi importante para que nós, como turma, começássemos a compreender que o Cálculo Numérico não consiste apenas em obter resultados, mas também em analisar sua precisão, confiabilidade e limitações. Para mim, essa foi uma das principais aprendizagens da aula.

Durante as atividades, consegui compreender melhor conceitos como erro absoluto, erro relativo, arredondamento, truncamento, propagação de erros e estabilidade numérica. Os exemplos práticos e a utilização do Python ajudaram a tornar esses conceitos mais claros para mim.

De maneira geral, considero que a aula estabeleceu uma base importante para os próximos conteúdos da disciplina. Ao final, entendi que, em Cálculo Numérico, não basta encontrar um resultado: também preciso saber avaliar e interpretar sua precisão.

✦ **Observação sobre a continuidade da aula:** O conteúdo trabalhado no **Anexo 01**, disponibilizado junto ao material da Aula 02, **será registrado no relatório da Aula 03**. A Aula 03 corresponde à retomada e continuação do conteúdo, mantendo os conceitos apresentados anteriormente e avançando para novas atividades.

Link do google colab: https://colab.research.google.com/drive/11RhNQEfs07T-cviueOJOZ8T7VC53Wc_a?usp=sharing